Lecture 18

Algebraic Data Types

CS 12 Team 25.2
A.Y. 2025–2026, 2nd Sem

Department of Computer Science
College of Engineering
University of the Philippines Diliman

Sanity Check

- Attendance?
- Audible at the back?
- TV(s) on?
  - (we’re working on getting the other TV fixed…)
- **Recording?**
- Important announcements?

Elm Resources

- References
  - https://guide.elm-lang.org/
  - https://elmprogramming.com/
  - https://elm-lang.org/examples
- Online editors
  - https://ellie-app.com/
  - https://elm-lang.org/try
  - https://elm-editor.com/

Motivating Problem: Data representation

Which type below represents BS CompSci student standing best?

- Freshman, Sophomore, Junior, Senior, Other

```
type Standing = Int
```

```
type Standing = String
```

```
type alias Standing =
  { isFreshman : Bool
  , isSophomore : Bool
```

```
    , isJunior : Bool
    , isSenior : Bool
    }
```

Aside: `type` vs `type alias`

- `type alias`: Gives a new name to an *existing type*
- `type`: Creates a *new type*

Types as Sets

Types can be defined as sets with values as set members

Elm type	Definition as set	Cardinality
`Bool`	{True, False}	2
`Int`	$[-2^{31}, 2^{31} - 1]$	2^{32}
`String`	{“”, “hello”, “CS 12”, ...}	∞
`(Bool, Bool)`	{(False,False),(False,True), (True,False),(True,True)}	4

Types as Sets

Composite type: Type composed of other types

- Python: Lists, dicts, sets, classes, data classes, unions, etc.
- Elm: Tuples, Lists, Arrays, Dicts, etc.

```
type alias R =
  { ateBreakfast: Bool
  , ateLunch: Bool
  }
```

Types as Sets

- Possible values:
  - `{ ateBreakfast = False, ateLunch = False}`
  - `{ ateBreakfast = False, ateLunch = True}`
  - `{ ateBreakfast = True, ateLunch = False}`
  - `{ ateBreakfast = True, ateLunch = True}`
- $|R| = 4$

Product Type

Product type: Type formed by composing other types with *AND*

- `type alias R = {ateBreakfast: Bool, ateLunch: Bool}`
  - `Bool` *AND* `Bool`
- Cardinality: Product of cardinalities of components
  - $|R| = |\text{Bool}| \times |\text{Bool}|$
  - $|R| = 2 \times 2$
  - $|R| = 4$

Union of Literal Types

```
Standing = Literal["Freshman"] | Literal["Sophomore"] |
Literal["Junior"] | Literal["Senior"] | Literal["Other"]
```

- Is `Standing` a product type?

Sum Type

Sum type: Type formed by composing other types with *OR*

- `Union` type in Python, tagged unions in Elm (more on this in a bit)
- `type R = Bool | Nothing`
  - `Bool` *OR* `Nothing`
- Cardinality: Sum of cardinalities of choices
  - $|R| = |\text{Bool}| + |\text{Nothing}|$
  - $|R| = |\text{Bool}| + 1$

- $|R| = 2 + 1$
- $|R| = 3$

Back to Motivating Problem

Which type below represents *BS CompSci student standing* best?

- `type Standing = number` ($|\text{Standing}| = \infty$)
- `type Standing = String` ($|\text{Standing}| = \infty$)

Back to Motivating Problem

Which type below represents *BS CompSci student standing* best?

- ```
  type Standing =
    { isFreshman: Bool
    , isSophomore: Bool
    , isJunior: Bool
    , isSenior: Bool
    }
  ```

  ($|\text{Standing}| = 8$)
- `type Standing = Freshman | Sophomore | Junior | Senior | Other` ($|\text{Standing}| = 5$)
  ```

Back to Motivating Problem

We prefer most accurate way to represent data (ideally isomorphic with actual set of values)

Algebraic Data Types

Algebraic data type: Type formed by composing other types with *AND* and/or *OR*

- Composed of *sum types* and *product types*
- Usually expressed using tagged unions

Algebraic Data Types

Tagged union: Union type with *tag* that identifies which type is in use

- Also called sum type, discriminated/disjoint union, choice type
- *Tag*: Data that can be used to identify which variant a tagged union value is

Algebraic Data Types

```
type Standing
    = Freshman
    | Sophomore
    | Junior
    | Senior
    | Other
```

What if `Other` needs to include total number of years?

Tagged Unions in Elm

Tagged union: Type that takes on exactly one of several variants

- Sum type

```
type DayOfWeek
    = Sunday
    | Monday
    | Tuesday
    | Wednesday
    | Thursday
    | Friday
    | Saturday
```

Tagged Unions in Elm

- Variants can also *carry data*:

```
type CrsSectionSlots
    = WithSlots Int
    | Full
    | Overbooked Int
    | Dissolved
```

```
type PageState
    = Loading
    | PageReady PageData    -- Record
    | PageError ErrorData   -- Record
```

Tagged Unions in Elm

- `Maybe` is implemented as a tagged union:

```
type Maybe a
    = Just a
    | Nothing
```

Tagged Unions in Elm

- Variants can be initialized:

Tagged Unions in Elm

```
type CrsSectionSlots
    = WithSlots Int
    | Full
    | Overbooked Int
    | Dissolved
type alias CrsSection =
    { course : String
    , section : String
    , capacity : Int
    , slots : CrsSectionSlots
    }
```

Tagged Unions in Elm

```
example =
    { course = "CS 12"
    , section = "TCD"
    , capacity = 80
    , slots = WithSlots 14 }
```

Pattern Matching

- `case` statement for destructuring variants:

```
toSlotStr : CrsSection -> String
toSlotStr section =
    let
        toFractionStr num den =
            String.fromInt num ++ " / " ++ String.fromInt den
    in
    case section.slots of
        WithSlots left ->
            "OPEN " ++ toFractionStr left section.capacity
        Full ->
```

```
            "FULL " ++ toFractionStr section.capacity
            section.capacity
        Overbooked over ->
            "OVERBOOKED " ++ toFractionStr over section.capacity
        Dissolved ->
            "DISSOLVED"
```

Pattern Matching

- _ as wildcard or to ignore unneeded carried data:

```
hasSlots : CrsSection -> Bool
hasSlots section =
    case section.slots of
        WithSlots _ ->
            True
        _ ->
            False
getWithSlots : List CrsSection -> ListCrsSection
getWithSlots sections =
    List.filter hasSlots sections
```

Data Modeling Example

Scenario: Food ordering application for a restaurant; food items as follows:

- *Tapsilog*
  - Should specify whether vinegar should be included
  - Should specify whether atchara should be included
- *Bottled water*
  - No additional specifications
- *Fries*
  - Should specify dip (ketchup or BBQ sauce)

- *Milk tea*
  - Should specify sugar level ($0\%, 25\%, 50\%, 75\%, 100\%$)

How do you represent `FoodItem`?

Tagged Union Representation

```
type Dip = Ketchup | BBQSauce
type SugarPercent = Percent0 | Percent25 | Percent50 | Percent75 |
Percent100

type FoodItem
    = Tapsilog { vinegar: Bool, atchara: Bool }
    | Water
    | Fries Dip
    | MilkTea SugarPercent
```

Exhaustiveness Checking

`calculateCost` computes the total to be shown to the customer; what is wrong with the implementation below?

```
calculateCost: List FoodItem -> Float
calculateCost items =
    let
        getItemCost: FoodItem -> Float
        getItemCost item =
            if item == Tapsilog then
                150
            else if item == Fries then
```

```
            120
        else if item == MilkTea then
            100
        else
            25
in
List.map getItemCost items
|> List.foldl (+) 0
```

Exhaustiveness Checking

- Syntax error!

```
calculateCost: List FoodItem -> Float
calculateCost items =
    let
        getItemCost: FoodItem -> Float
        getItemCost item =
            if item == Tapsilog then
                150
            else if item == Fries then
                120
            else if item == MilkTea then
```

```
                100
            else
                25
    in
    List.map getItemCost items
    |> List.foldl (+) 0
```

Exhaustiveness Checking

- Pattern matching allows unwrapping of tagged unions and exhaustiveness checking:

```
calculateCost: List FoodItem -> Float
calculateCost items =
    let
        getItemCost: FoodItem -> Float
        getItemCost item =
            case item of
                Tapsilog _ ->
                    150
```

```
                Fries _ ->
                    120
                MilkTea _ ->
                    100
                Water ->
                    25
    in
    List.map getItemCost items
    |> List.foldl (+) 0
```

Exhaustiveness Checking

- Elm can detect if there is a possible branch missing:

```
getItemCost: FoodItem -> Float
getItemCost item =
    case item of
        Tapsilog _ ->
            150
        Fries _ ->
            120
        MilkTea _ ->
            100
```